

## *Chapter 14: Cross-validation*

*Shravan Vasishth*

2026-03-31

### *Contents*

|           |                                                                  |           |
|-----------|------------------------------------------------------------------|-----------|
| <i>1</i>  | <i>Textbook</i>                                                  | <i>2</i>  |
| <i>2</i>  | <i>Introduction</i>                                              | <i>2</i>  |
| <i>3</i>  | <i>The expected log predictive density of a model</i>            | <i>3</i>  |
| <i>4</i>  | <i>K-fold and leave-one-out cross-validation</i>                 | <i>5</i>  |
| <i>5</i>  | <i>Testing the N<sub>400</sub> effect using cross-validation</i> | <i>8</i>  |
| <i>6</i>  | <i>Cross-validation with PSIS-LOO</i>                            | <i>9</i>  |
| <i>7</i>  | <i>Cross-validation with K-fold</i>                              | <i>12</i> |
| <i>8</i>  | <i>Leave-one-group-out cross-validation</i>                      | <i>14</i> |
| <i>9</i>  | <i>Comparing different likelihoods with cross-validation</i>     | <i>15</i> |
| <i>10</i> | <i>Issues with cross-validation</i>                              | <i>20</i> |
| <i>11</i> | <i>Cross-validation in Stan</i>                                  | <i>22</i> |
| <i>12</i> | <i>PSIS-LOO-CV in Stan</i>                                       | <i>23</i> |
| <i>13</i> | <i>K-fold-CV in Stan</i>                                         | <i>26</i> |
| <i>14</i> | <i>Summary</i>                                                   | <i>30</i> |
| <i>15</i> | <i>Further reading</i>                                           | <i>30</i> |

## 1 Textbook

Introduction to Bayesian Data Analysis for Cognitive Science

Nicenboim, Schad, Vasishth

- Online version: <https://bruno.nicenboim.me/bayescogsci/>
- Source code: <https://github.com/bnicenboim/bayescogsci>
- Physical book: [here](#)

**Be sure to read the textbook's chapter 14 in addition to watching this lecture.**

## 2 Introduction

A popular way to evaluate and compare models is to investigate their ability to make predictions for “out-of-sample data,” that is, to use what we learned from the observed data to predict future or unseen observations. Cross-validation is used to test which of the models under consideration is/are able to learn the most from our data in order to make better predictions. In cognitive science, the goal is only rarely to predict future observations; more often, the researcher wants to compare the performance of different models given some observed data. Nevertheless, there are situations in cognitive science where evaluating the predictive performance of models becomes important.

The objective of cross-validation is to avoid over-optimistic predictions; such over-optimistic predictions would arise if we were to use the data to estimate the parameters of our model, and then use these estimates to predict the same data. That amounts to using the data twice. The basic idea behind cross-validation is that the models are fit with a large subset of the data, the *training set*, and then the fitted models are used to predict a smaller, unseen part of the data, the *held-out set*. In order to treat the entire data set as a held-out set and to evaluate the predictive accuracy using every observation, one successively changes what constitutes the training set and the held-out

set by refitting the model and evaluating it in different folds. This ensures that the predictions of the model are tested over the entire data set.

### 3 The expected log predictive density of a model

In order to compare the quality of the posterior predictions of two models, a *utility* function or a *scoring rule* is used. The logarithmic score rule, shown in the Equation below:

$$u(\mathcal{M}_1, y_{pred}) = \log p(y_{pred} | y, \mathcal{M}_1) \quad (1)$$

has been proposed as a reasonable way to assess the posterior predictive distribution of a candidate model  $\mathcal{M}_1$  given the data  $y$ . This approach is reasonable because it takes into account the uncertainty of the predictions (unlike, for example, using the mean square error). If new observations are well-accounted by the posterior predictive distribution, then the density of the posterior predictive distribution is high and so is its logarithm.

Unlike the Bayes factor, the prior is absent from the Equation shown above. However, the prior does have a role here: The posterior predictive distribution is based on the posterior distribution  $p(\Theta | y)$  (where  $\Theta$  is a vector of all the parameters of the model), which, according to Bayes' rule, depends on both priors and likelihood together. Recall this Equation, repeated here for convenience:

$$p(y_{pred} | y) = \int_{\Theta} p(y_{pred} | \Theta) p(\Theta | y) d\Theta \quad (2)$$

Here, we are implicitly conditioning on the model under consideration:

$$p(y_{pred} | y, \mathcal{M}_1) = \int_{\Theta} p(y_{pred} | \Theta, \mathcal{M}_1) p(\Theta | y, \mathcal{M}_1) d\Theta \quad (3)$$

The predicted data,  $y_{pred}$ , are unknown to the utility function, so the utility function as presented in the first

Equation above cannot be evaluated. For this reason, we marginalize over *all possible future data* (calculating  $E[\log p(y_{pred}|y, \mathcal{M}_1)]$ ); this expression is called the expected log predictive density of model  $\mathcal{M}_1$ :

$$elpd = u(\mathcal{M}_1) = \int_{y_{pred}} p_t(y_{pred}) \log p(y_{pred} | y, \mathcal{M}_1) dy_{pred} \quad (4)$$

where  $p_t$  is the true data generating distribution. If we consider a set of models, the model with the highest *elpd* is the model with the predictions that are the closest to those of the true data generating process.<sup>1</sup> The intuition behind the Equation above is that we are evaluating the predictive distribution of  $\mathcal{M}_1$  over all possible future data weighted by how likely the future data are according to their true distribution. This means that observations that are very likely according to the true model will have a higher weight than unlikely ones.

But we don't know the true data-generating distribution,  $p_t$ ! If we knew it, we wouldn't be looking for the best model, since we would already know what the best model is.

We can use the observed data distribution as a proxy for the true data generating distribution. So instead of weighting the predictive distribution by the true density of all possible future data, we just use the  $N$  observations that we have. We can do that because our observations are presumed to be samples from the true distribution of the data: Under this assumption, observations with higher likelihood according to the true distribution of the data will also be obtained more often. This means that instead of integrating, we sum the posterior predictive density of the observations and we give to each observation the same weight; this is valid because observations that are appearing more often in the data should presumably also appear more often in the future. This quantity is called *log pointwise predictive density* or *lpd* [Vehtari and colleagues write this without the  $1/N$ ]:

<sup>1</sup> Maximizing the *elpd* in the Equation above is also equivalent to minimizing the Kullback–Leibler (KL) divergence from the true data generating distribution  $p_t(y_{pred})$  to the *posterior predictive distribution* of the candidate model  $\mathcal{M}_1$ .

$$lpd = \frac{1}{N} \sum_{n=1}^N \log p(y_n | y, \mathcal{M}_1) \quad (5)$$

The  $lpd$  is an overestimate of  $elpd$  for actual future data, because the parameters of the posterior predictive distribution are estimated with the same observations that we are considering out-of-sample. Incidentally, this also explains why posterior predictive checks are generally optimistic and good fits cannot be taken too seriously. But they do serve the purpose of identifying very strong model misspecifications.<sup>2</sup>

However, we can obtain a more conservative estimate of the predictive performance of a model using cross-validation. This is explained next. [As an aside, we mention here that there are also other alternatives to cross-validation; see the reference in the textbook].

<sup>2</sup> The double use of the data is also a problem when one relies on information criteria like the Akaike Information Criterion (AIC) or the Bayesian Information Criterion (BIC).

#### 4 *K-fold and leave-one-out cross-validation*

The basic idea of K-fold cross-validation (K-fold-CV) is to split the  $N$  observations of our data into  $K$  subsets, such that each subset is used as a validation (or held-out) set,  $D_k$ , while the remaining set (the training set),  $D_{-k}$  is used for estimating the parameters and approximating  $p_t$ , the true data distribution. The leave-one-out cross-validation (LOO-CV) method represents a special case of K-fold-CV where,  $K = N$  and the training set only excludes one observation. For the general case, K-fold-CV, we estimate  $elpd$  as follows.

$$\widehat{elpd} = \frac{1}{N} \sum_{n=1}^N \log p(y_n | D_{-k}, \mathcal{M}_1) \text{ with } y_n \in D_k \quad (6)$$

In the above Equation, each observation,  $y_n$ , belongs to a certain “validation” fold,  $D_k$ , and the predictive accuracy of  $y_n$  is evaluated based on a posterior predictive model trained on the set,  $D_{-k}$ , which is the complete data set excluding the validation fold that contains the  $n$ -th

observation. This means that the posterior predictive distribution used to evaluate  $y_n$  was derived without having information from that  $y_n$ -th observation (in other words, the model was trained without that observation in the subset of the data,  $D_{-k}$ ).

In K-fold-CV, several observations are held out in the same (validation) fold. This means that the held-out observations are split among K folds, and  $D_{-k}$ , the data used to derive the posterior predictive distribution, contain only a proportion of the observations; this proportion is  $(1 - 1/K)$ . By contrast, in leave-one-out cross-validation, the held-out data set includes only one observation. That is,  $D_{-k}$  contains the entire data set except for one data point,  $y_n$ , with  $n = 1, \dots, N$ , that is  $D_{-n}$ . An online section explains the algorithm in detail.

For the general case of K-fold-CV, Vehtari et al 2017 define the expected log *pointwise* predictive density of the observation  $y_n$  as follows:

$$\widehat{elpd}_n = \log p(y_n | D_{-k}, \mathcal{M}_1) \text{ with } y_n \in D_k \quad (7)$$

This quantity indicates the predictive accuracy of the model  $\mathcal{M}_1$  for a single observation, and it is reported in the package `loo`, which also `brms` relies on. In addition, the `loo` package uses the sum of the expected log pointwise predictive density,  $\sum \widehat{elpd}_n$  without  $\frac{1}{N}$ ) as a measure of predictive accuracy (this is referred to as `elpd_loo` or `elpd_kfold` in the packages `loo` and `brms`). For model comparison, the difference between the  $\sum \widehat{elpd}_n$  of competing models can be computed, including the standard deviation of the sampling distribution of the difference. It's important to notice that we are calculating an approximation to the expectation  $elpd$  that we actually want to compute, and thus we always need to consider its inherent randomness (due to having a limited amount of data), which is quantified by the standard error.

Unlike what is common with information criterion methods (such as the Akaike Information Criterion, AIC, and the Deviance Information Criterion, DIC), a higher

$\widehat{elpd}$  means higher predictive accuracy. An alternative to using  $\widehat{elpd}$  is to examine  $-2 \times \widehat{elpd}$ , which is equivalent to deviance, and is called the LOO Information Criterion (LOOIC).

The approximation to the true data generating distribution is worse when fewer observations are used, and thus ideally we would set  $K = N$ , and thus compute LOO-CV rather than K-fold-CV. The main advantage of LOO-CV is its robustness, since the training set is as similar as possible to the observed data, and the same observations are never used simultaneously for training and evaluating the predictions. A major disadvantage is the computational burden, since we need to fit a model as many times as the number of observations. The package `loo` provides an approximation to LOO-CV, Pareto smoothed importance sampling leave-one-out [PSIS-LOO] which, as we show next, is relatively straightforward to use in `brms` and in Stan models (see <https://mc-stan.org/loo/articles/loo2-with-rstan.html>). However, in some cases, its estimates can be unreliable; this is indicated by the estimated shape parameter  $\hat{k}$  of the generalized Pareto distribution (<https://mc-stan.org/loo/reference/pareto-k-diagnostic.html>). The value  $\hat{k}$  (which is unrelated to the  $k$  in K-fold-CV) estimates how far an approximated individual leave-one-out distribution is from the full distribution. If leaving out an observation substantially alters the posterior, then importance sampling cannot provide a reliable estimate. Very high  $\hat{k}$  values often indicate model misspecification, outliers, or mistakes in data processing. The threshold for  $\hat{k}$  depends on the sample size and it is reported by the `loo()` function. In cases where one or several point-wise predictive density have associated large  $\hat{k}$ , either (i) the problematic predictions can be refitted with exact LOO-CV, (ii) one can try some additional computations using the existing posterior sample based on the moment matching approximation [see <https://mc-stan.org/loo/articles/loo2-moment-matching.html>], or (iii) one can

abandon PSIS-LOO-CV and use K-fold-CV, with K typically set to 10.

One of the main disadvantages of cross-validation (in comparison with Bayes factor at least) is that the numerical difference in predictive accuracy is hard to interpret. As a rule of thumb, it has been suggested that if the elpd difference (`elpd_diff` in the `loo` package) is less than 4, the difference is small, and if it is larger than 4, one should compare that difference to its standard error (`se_diff`).

### 5 *Testing the N<sub>400</sub> effect using cross-validation*

As in the Bayes factor chapter, let us revisit the effect of cloze probability on the N<sub>400</sub> average signal was estimated. Consider two models here, a model that includes the effect of cloze probability, such as `fit_N400_sih`, and a null model.

Verify the model that was fit; this is a hierarchical model that includes an effect of cloze probability:

```
library(bcogsci)
data("df_eeg")
df_eeg$c_cloze <- df_eeg$cloze - mean(df_eeg$cloze)

prior_sih_full <-
  c(prior(normal(0, 10), class = Intercept),
    prior(normal(0, 10), class = b, coef = c_cloze),
    prior(normal(0, 50), class = sigma),
    prior(normal(0, 20),
      class = sd, coef = Intercept,
      group = subj),
    prior(normal(0, 20),
      class = sd, coef = c_cloze,
      group = subj),
    prior(lkj(2), class = cor, group = subj),
    prior(normal(0, 20),
      class = sd, coef = Intercept,
      group = item),
    prior(normal(0, 20),
```



```

      class = sd, coef = c_cloze,
      group = item),
prior(lkj(2), class = cor, group = item))
fit_N400_sih <- brm(n400 ~ c_cloze + (c_cloze | subj) +
                    (c_cloze | item),
                    prior = prior_sih_full,
                    data = df_eeg)

```

In contrast to the situation with Bayes factor, priors are less critical for cross-validation. Priors are only important in cross-validation to the extent that they affect parameter estimation: As discussed earlier, very narrow priors can bias the posterior; and unrealistically wide priors can lead to convergence problems. The number of samples is also less critical than with Bayes factor; most of the uncertainty in the estimates of the  $\widehat{elpd}$  is due to the number of observations. However, a very small number of samples can affect the  $\widehat{elpd}$  because the posterior estimation will be affected by the small sample size. We update our previous formula to define a null model as follows:

```
fit_N400_sih_null <- update(fit_N400_sih, ~ . - c_cloze)
```

## 6 Cross-validation with PSIS-LOO

Estimating  $elpd$  using PSIS-LOO is very straightforward with `brms`, which uses the package `loo` as a back-end. There is no need to refit the model, and `loo` takes care of applying the PSIS approximation to derive estimates and standard errors.

```

(loo_sih <- loo(fit_N400_sih))

##
## Computed from 4000 by 2863 log-likelihood matrix.
##
##           Estimate    SE
## elpd_loo -11093.1  46.7
## p_loo      81.4   2.7
## looic      22186.2 93.4

```

```
## -----
## MCSE of elpd_loo is 0.2.
## MCSE and ESS estimates assume MCMC draws (r_eff in [0.4, 1.5]).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.

(loo_sih_null <- loo(fit_N400_sih_null))

##
## Computed from 4000 by 2863 log-likelihood matrix.
##
##           Estimate   SE
## elpd_loo -11095.4 46.5
## p_loo      89.4  3.0
## looic     22190.9 93.1
## -----
## MCSE of elpd_loo is 0.2.
## MCSE and ESS estimates assume MCMC draws (r_eff in [0.4, 1.7]).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.
```

The function `loo` reports three quantities with their standard error:

1. `elpd_loo` is the sum of pointwise predictive accuracy (a larger, less negative number indicates better predictions).
2. `p_loo` is an estimate of effective complexity of the model; asymptotically and under certain regularity conditions, `p_loo` can be interpreted as the effective number of parameters. If `p_loo` is larger than the number of data points or parameters, this may indicate a severe model misspecification.
3. `looic` is simply  $-2 \times \text{elpd\_loo}$ , the *elpd* on the deviance scale. This is called the information criterion, and is mainly provided for historical reasons: other information criteria like the AIC (Akaike Information Criterion) and the DIC (Deviance Information Criterion) are commonly used in model selection.

It's important to bear in mind that the PSIS-LOO approximation to LOO can only be trusted if there are no large Pareto ( $\hat{k}$ ) estimates. To compare the models, take a look at the difference between `elpd_loo` and the standard error of that difference:

```
loo_compare(loo_sih, loo_sih_null)

##               elpd_diff se_diff
## fit_N400_sih      0.0      0.0
## fit_N400_sih_null -2.4      2.5
```

Although the model that includes cloze probability as a predictor has higher predictive accuracy, the difference is smaller than 4 and it's smaller than two SE. This means that from the perspective of LOO-CV, both models are almost indistinguishable! In fact, the same will happen if the model is compared using logarithmic predictability to the linear or null model; there is an exercise on this in the homework.

It is also possible to check whether the alternative model is making good predictions for some range of values by examining the difference in pointwise predictive accuracy as a function of, for example, cloze probability. In the following plot, we subtract the predictive accuracy of the alternative model from the accuracy of the null model; larger differences can be interpreted as an advantage for the alternative model. However, as far as posterior predictive accuracy goes, both models are quite similar. The Figure below shows that the difference in predictive accuracy is symmetrical with respect to the zero; as we go further from the mean cloze (which is around 0.5), the differences in predictions are larger but they span positive and negative values.

The following code stores the difference in predictive accuracy of the models in a variable and plots it.

```
df_eeg <- mutate(df_eeg,
  diff_elpd = loo_sih$pointwise[, "elpd_loo"] -
    loo_sih_null$pointwise[, "elpd_loo"])
```

```
ggplot(df_eeg, aes(x = cloze, y = diff_elpd)) +
  geom_point(alpha = .4, position = position_jitter(w = .001, h = 0))
```

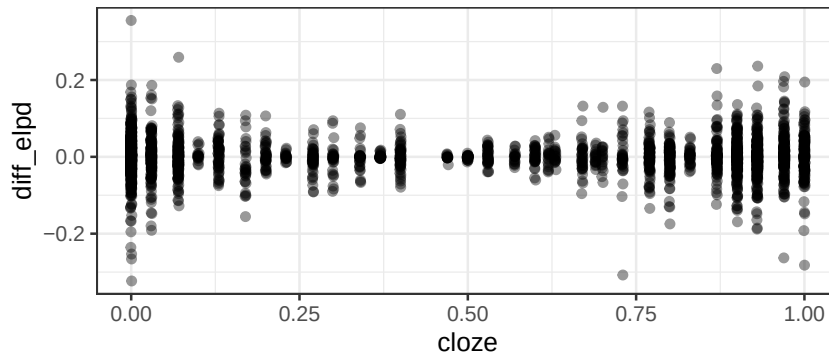


Figure 1: The difference in predictive accuracy between a model including the effect of cloze and a null model. A larger (more positive) difference indicates an advantage for the model that includes the effect of cloze.

The expectation was that, similar to the Bayes factor, cross-validation techniques will also show that a model that includes cloze probability as a predictor is superior to a model without it. It is unsettling that the above result does not show this: the effect of cloze probability on the  $N_{400}$  has been replicated in numerous studies.

Before discussing why a large difference was not observed, let us check what K-fold-CV yields.

## 7 Cross-validation with K-fold

Estimating  $elpd$  using k-fold-CV has the advantage of omitting one layer of approximations: the  $elpd$  based on PSIS-LOO-CV is an approximation of the  $elpd$  based on *exact* LOO-CV (and we saw how any cross-validation approach gave us an approximation to the true  $elpd$ ). This means that we don't need to worry about  $\hat{k}$ . However, K-fold also uses a reduced training set in comparison with LOO, worsening the approximation to the true generating process  $p_t$ .

Before dividing our data into folds, we need to think about the way the data will be split: The data could be split randomly, but this approach would have the risk that in some of the training sets, observations from a particular subject, for example, could be completely absent. Such

an omission will lead to large differences in predictive accuracy between folds. This situation can be avoided by using stratification: split the observations into groups, ensuring that relative category frequencies are approximately preserved in the training and held-out data. This stratification can be achieved by using the `kfold()` function, available in the package `brms`, by setting `folds = "stratified"` and `group = "subj"`, by default `K` is set to 10, but that can be changed. Additionally, since we know the models converge, we use only one chain for each model and parallelize the fitting procedure with the `plan(multisession)` function from the `future` package, returning to the default setting with `plan(sequential)`.

```
library(future)
plan(multisession, workers = 4)
kfold_sih <- kfold(fit_N400_sih,
                  folds = "stratified",
                  group = "subj",
                  chains = 1)
kfold_sih_null <- kfold(fit_N400_sih_null,
                      folds = "stratified",
                      group = "subj")
plan(sequential)
```

Running K-fold CV takes some time since each model is re-fit `K` times. Inspect the *elpd* values:

```
kfold_sih

##
## Based on 10-fold cross-validation.
##
##           Estimate    SE
## elpd_kfold -11097.1 47.0
## p_kfold      85.4   3.8
## kfoldic     22194.3 94.0

kfold_sih_null

##
```

```
## Based on 10-fold cross-validation.
##
##           Estimate   SE
## elpd_kfold -11095.4 46.3
## p_kfold      89.4   3.6
## kfoldic      22190.9 92.5
```

Compare the two models using `loo_compare` (this function is used for both PSIS-LOO-CV and K-fold-CV):

```
loo_compare(kfold_sih, kfold_sih_null)

##           elpd_diff se_diff
## fit_N400_sih_null  0.0      0.0
## fit_N400_sih      -1.7      4.3
```

Because the  $\widehat{elpd}$  values depend on how the folds were formed, K-fold CV introduces an additional element of randomness compared to (PSIS-)LOO-CV. To verify the robustness of the results, one can rerun the `kfold()` function so that different random configurations of the folds are selected. In this case, however, the results with K-fold-CV and PSIS-LOO-CV are quite similar: The two models can't really be distinguished.

## 8 Leave-one-group-out cross-validation

An alternative to splitting the observations randomly using stratification is to treat naturally occurring clusters as folds; this is leave-one-group-out cross-validation (LOGO-CV). The output of LOGO-CV tells us about the capacity of the models for generalizing to unseen clusters. LOGO-CV is implemented next using subjects as the group of interest.

In general, there is some tension between the arguments for K-fold-CV with folds stratified by subject, which ensures data from all subjects in each fold, and for LOGO-CV, which isolates data from different subjects. However, the two approaches have different goals and merits. The first approach answers how well the model

generalizes to unseen data from this particular set of subjects. The second approach addresses how well the model generalizes to the behavior of unseen subjects if we were to collect more data.

```
plan(multisession, workers = 4)
logo_sih <- kfold(fit_N400_sih,
                  group = "subj",
                  chains = 1)
logo_sih_null <- kfold(fit_N400_sih_null,
                       group = "subj",
                       chains = 1)
plan(sequential)
```

Running LOGO-CV with subjects takes even longer since each model is re-fit as many times as there are subjects, in this case 37 times. We can now inspect the *elpd* estimates and evaluate which model generalizes better to unseen subjects.

Compare the models using `loo_compare`.

```
loo_compare(logo_sih, logo_sih_null)

##               elpd_diff se_diff
## fit_N400_sih      0.0      0.0
## fit_N400_sih_null -3.3      3.3
```

As before, and as with PSIS-LOO-CV and with K-fold-CV, the two models can't be distinguished. Even though the full model and the null model presented earlier make different predictions, the difference in predictive accuracy is not substantial compared to the variability in the data. In these situations, cross-validation is not very useful for distinguishing very small effect sizes from zero effect sizes; see the section on issues below.

## 9 Comparing different likelihoods with cross-validation

One can also compare two models with different likelihoods. Chapter 3 showed how a log-normal distribution

was a more appropriate likelihood than a normal distribution for response times data. This was because response times are bounded by zero and right skewed, unlike the symmetrical normal distribution. Let's use PSIS-LOO-CV to compare the predictive accuracy of the Stroop model from chapter 5, which assumed a log-normal likelihood to fit response times of correct responses with a similar model which assumes a normal likelihood. We can only compare models fit to the same data; we can't compare models fit to different dependent variables (e.g., one raw dependent variable and one log-transformed).

Load the data from `bcogsci`, create a sum-coded predictor, and fit the model as before.

```
data("df_stroop")
df_stroop <- df_stroop %>%
  mutate(c_cond = if_else(condition == "Incongruent", 1, -1))

fit_stroop <- brm(RT ~ c_cond + (c_cond | subj),
  family = lognormal(),
  prior =
    c(prior(normal(6, 1.5), class = Intercept),
      prior(normal(0, 0.01), class = b),
      prior(normal(0, 1), class = sigma),
      prior(normal(0, 1), class = sd),
      prior(lkj(2), class = cor)),
  iter=5000,
  data = df_stroop)

fit_stroop_log <- brm(RT ~ c_cond + (c_cond | subj),
  family = lognormal(),
  prior = c(prior(normal(6, 1.5), class = Intercept),
    prior(normal(0, 1), class = b),
    prior(normal(0, 1), class = sigma),
    prior(normal(0, 1), class = sd),
    prior(lkj(2), class = cor)),
  data = df_stroop)
```

Calculate the  $elpd_{loo}$  for the original model with the log-normal likelihood:



```

loo_stroop_log <- loo(fit_stroop_log)

loo_stroop_log

##
## Computed from 4000 by 3058 log-likelihood matrix.
##
##           Estimate      SE
## elpd_loo -19858.8   93.5
## p_loo      60.7    4.0
## looic      39717.7 187.1
## -----
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume MCMC draws (r_eff in [0.4, 2.4]).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.

```

The summary shows that  $\hat{k}$  estimates are ok.

Now fit a similar model which assumes that the likelihood is a normal distribution. It's important now to change the priors since they are on a different scale (namely, in milliseconds). We choose reasonable but wide priors. A sensitivity analysis can be done if we are unsure about the priors. However, unlike what happened with the Bayes factor, the priors are going to affect cross-validation based model comparison only as far as they have a noticeable effect on the posterior distribution.

```

fit_stroop_normal <- brm(RT ~ c_cond + (c_cond | subj),
  family = gaussian(),
  prior = c(prior(normal(400, 600), class = Intercept),
    prior(normal(0, 100), class = b),
    prior(normal(0, 300), class = sigma),
    prior(normal(0, 300), class = sd),
    prior(lkj(2), class = cor)),
  data = df_stroop)

```

The estimated  $elpd$  based on PSIS-LOO-CV has several large  $\hat{k}$  values.

```

loo_stroop_normal <- loo(fit_stroop_normal)

```

```
loo_stroop_normal

##
## Computed from 4000 by 3058 log-likelihood matrix.
##
##           Estimate      SE
## elpd_loo -21591.0 482.7
## p_loo      117.5  68.7
## looic      43182.1 965.5
## -----
## MCSE of elpd_loo is NA.
## MCSE and ESS estimates assume MCMC draws (r_eff in [0.5, 2.7]).
##
## Pareto k diagnostic values:
##
##              Count Pct.    Min. ESS
## (-Inf, 0.7] (good)   3055 99.9%    55
## (0.7, 1]   (bad)      2   0.1%   <NA>
## (1, Inf)   (very bad) 1   0.0%   <NA>
## See help('pareto-k-diagnostic') for details.
```

We use exact LOO (rather than its approximation) for the problematic observations. By setting `reloo = TRUE`, the 3 problematic observations with  $\hat{k}$  values over 0.7 are re-fit using exact LOO-CV.<sup>3</sup>

```
loo_stroop_normal <- loo(fit_stroop_normal, reloo = TRUE)
loo_stroop_normal

##
## Computed from 4000 by 3058 log-likelihood matrix.
##
##           Estimate      SE
## elpd_loo -21704.9 592.9
## p_loo      231.4 182.1
## looic      43409.7 1185.8
## -----
## MCSE of elpd_loo is NaN.
## MCSE and ESS estimates assume MCMC draws (r_eff in [0.5, 2.7]).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.
```

<sup>3</sup> An alternative approach is to use the model matching approximation for problematic observations, by setting `moment_match = TRUE` in the `loo()` call (and we also need to fit the model with `save_pars = save_pars(all = TRUE)`). In this particular case, this approximation won't solve our problem.

Next, compare the models.

```
loo_compare(loo_stroop_log, loo_stroop_normal)
```

```
##               elpd_diff se_diff
## fit_stroop_log      0.0      0.0
## fit_stroop_normal -1846.0    537.4
```

Here, cross-validation shows a clear advantage for the model with the log-normal likelihood. The Figure below (sub-figure a) visualizes the pointwise predictive accuracy.

```
df_stroop <- df_stroop %>%
  mutate(diff_elpd = loo_stroop_log$pointwise[, "elpd_loo"] -
           loo_stroop_normal$pointwise[, "elpd_loo"])
ggplot(df_stroop, aes(x = RT, y = diff_elpd)) +
  geom_point(alpha = .4) + xlab("RC (ms)")
```

The sub-figure a shows that at first glance, the advantage of the log-normal likelihood seems to lie in being able to capture extremely slow observations. Sub-figure b zooms in to visualize the pointwise predictive accuracy for observations with response times smaller than two seconds.

```
ggplot(df_stroop, aes(x = RT, y = diff_elpd)) +
  geom_point(alpha = .3) +
  xlab("RC (ms)") +
  geom_hline(yintercept = 0, linetype = "dashed") +
  coord_cartesian(xlim = c(0, 2000), ylim = c(-10, 10))
```

(ref:diffpredacclog12) (a) Difference in predictive accuracy between Stroop models with log-normal and normal likelihoods, where a more positive value favors the log-normal model. (b) Difference in predictive accuracy between the same models, focusing on observations under two seconds, with positive values again favoring the log-normal model.

Sub-figure b suggests that the advantage of the log-normal likelihood seems to lie in being able to account for most of the observations in the data set, which occur around the 500 ms.

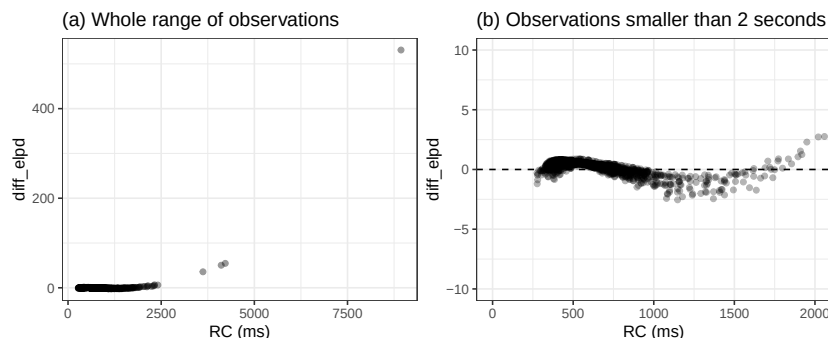


Figure 2: (ref:diffpredacclog12)

## 10 Issues with cross-validation

Sivula et al 2020 analyzed the behavior of the uncertainty estimate of the  $elpd$  in typical situations. Although they focus on LOO-CV, the consequences are the same for K-fold-CV (and cross-validation in a non-Bayesian context). Sivula et al identified three cases where the uncertainty estimates can perform badly:

1. The models make very similar predictions.
2. The number of observations is small.
3. The models are misspecified with outliers (influential extreme values) in the data.

When the models make similar predictions (as it is the case with nested models in earlier model comparisons), and when there is not much difference in the predictive performance of the models, the uncertainty estimates will behave badly. In these situations, cross-validation is not very useful for separating very small effect sizes from zero effect sizes. In addition, small differences in the predictive performance cannot reliably be detected by cross-validation if the number of observations is small. However, if the predictions are very similar, Sivula et al show that the same problems persist even with a larger data set.

One of the issues that cross-validation methods face when they are used to compare nested models lies in the way that the exact  $elpd$  is approximated: In cross-validation approximations, out-of-sample observations

are used, which are not part of the model that was fit. Every time the predictive accuracy of an observation is evaluated, modeling assumptions are ignored. One of the weaknesses of cross-validation is the high variance in the approximation of the integral over the unknown true data distribution,  $p_t$  [section 4 of Vehtari et al 2012].

Cross-validation methods are sometimes criticized because when a lot of data are available, they will give undue preference to the complex model in comparison to a true simpler model [Gronau et al 2018]. This might be true for toy examples using simulated data, where it is possible to obtain essentially unlimited observations, and where a model that is known to be wrong is compared with the true model.<sup>4</sup> However, the problems that occur in practice are often very different: This is because the true model is unknown and very likely not even under consideration in the model comparison [see Navarro 2018]. In our experience, in practical settings we are very far from the asymptotic behavior of cross-validation whereby it gives undue preference to a more complex model in comparison to a true simpler model. The main weakness of cross-validation is that, as mentioned above, modeling assumptions are ignored, which prevents it from selecting a more complex model rather than a simple one when there is only a modest gain in predictive fit.

An alternative to the cross-validation approach discussed here for nested models is the projection predictive method. However, this approach (which is less general since it is valid only for generalized linear models) has a somewhat different objective. In the projection predictive method, we first build the most complete predictive model, the *reference model*, and then we look for a simpler model that gives as similar predictions as the reference model. The idea is that for a given complexity (number of predictors), the model with the smallest predictive discrepancy to the reference model should be selected. See <https://github.com/stan-dev/projpred> for an imple-

<sup>4</sup> If the true model is under consideration among the models being compared, we are under an  $M_{closed}$  scenario. However, this is rarely realistic. The most common case is an  $M_{open}$  scenario [see the Bernardo and Smith book], where the true model is not included in the set of models being compared.

mentation of this approach. Thus this approach focuses on model simplification rather than on model comparison.

For models that are badly misspecified, the bias in the uncertainty makes their comparison unreliable as well. In this case, posterior predictive checks and possible model refinements are worth considering before carrying out model comparison.

If there is a large number of observations and the models under consideration are different enough from each other, the differences in predictive accuracy will dwarf the variance in the estimate of  $elpd$ , and cross-validation can be very useful. An example of this situation appeared earlier. When models are very different, one advantage of cross-validation methods in comparison with the Bayes factor is that the selection of priors is less critical in cross-validation. It is sometimes hard to decide on priors that encode our knowledge for one model, and this difficulty is exacerbated when we want to assign comparable prior information to models with a different number of parameters that might be on a different scale. Given that cross-validation methods are less sensitive to prior specification, different models can be compared on the same footing. Another difficulty with Bayes factors' dependency on priors is that different researchers may disagree on the priors, and selecting an uninformative prior to encompass these different views as a middle ground is not a neutral option for the Bayes factor, while it will barely affect cross-validation. See Nicenboim et al 2018 and Yadav et al 2022 for examples from psycholinguistics where K-fold-CV does help in distinguishing between models.

## 11 *Cross-validation in Stan*

PSIS-LOO-CV and K-fold-CV can also be used with our Stan models, but care has to be taken to store the appropriate log-likelihood in the generated quantities block.<sup>5</sup>

<sup>5</sup> This means the sampling notation should not be used in this block.

## 12 PSIS-LOO-CV in Stan

As explained earlier, PSIS-LOO (as implemented in the package `loo`) approximates the likelihood of the held-out data based on the observed data: it's faster (because only one model is fit), and it only requires a minimal modification of the Stan code we need to fit a model. By default, Stan only saves the sum of the log likelihood of each observation (in the parameter `lp__`). The log-likelihood of each observation needs to be stored in the generated quantities block.

We revisit the model implemented earlier, which was evaluated using the Bayes factor. Now, we want to compare the predictive performance of a model that assumes an effect of attentional load on pupil size against a similar model that assumes no effect. To do this, we assume the following likelihood:

$$p_{\text{size}_n} \sim \text{Normal}(\alpha + c_{\text{load}_n} \cdot \beta_1 + c_{\text{trial}} \cdot \beta_2 + c_{\text{load}} \cdot c_{\text{trial}} \cdot \beta_3, \sigma) \quad (8)$$

Define priors for all the  $\beta$ 's as before:

$$\begin{aligned} \alpha &\sim \text{Normal}(1000, 500) \\ \beta_{\{1,2,3\}} &\sim \text{Normal}(0, 100) \\ \sigma &\sim \text{Normal}_+(0, 1000) \end{aligned} \quad (9)$$

Prepare the data as before:

```
df_pupil <- df_pupil %>%
  mutate(c_load = load - mean(load),
         c_trial = trial - mean(trial))
ls_pupil <- list(c_load = df_pupil$c_load,
               c_trial = df_pupil$c_trial,
               p_size = df_pupil$p_size,
               N = nrow(df_pupil))
```

Add a generated quantities block to the model shown below. (It is also possible to run this block in a standalone file with the `rstan` function `gqs()`). If the variable name `log_lik` is used in the Stan code, the `loo` package

will know where to find the log likelihood of the observations.

Code the effects as `beta1`, `beta2`, `beta3` to more easily compare the model with the one used in the BF chapter, but in this case we could have used a vector or an array instead. This is the model `pupil_cv.stan` shown below:

```
data {
  int<lower = 1> N;
  vector[N] c_load;
  vector[N] c_trial;
  vector[N] p_size;
}
parameters {
  real alpha;
  real beta1;
  real beta2;
  real beta3;
  real<lower = 0> sigma;
}
model {
  // priors including all constants
  target += normal_lpdf(alpha | 1000, 500);
  target += normal_lpdf(beta1 | 0, 100);
  target += normal_lpdf(beta2 | 0, 100);
  target += normal_lpdf(beta3 | 0, 100);
  target += normal_lpdf(sigma | 0, 1000)
    - normal_lccdf(0 | 0, 1000);
  target += normal_lpdf(p_size | alpha + c_load * beta1 +
    c_trial * beta2 +
    c_load .* c_trial * beta3, sigma);
}
generated quantities{
  array[N] real log_lik;
  for (n in 1:N){
    log_lik[n] = normal_lpdf(p_size[n] | alpha + c_load[n] * beta1 +
      c_trial[n] * beta2 +
      c_load[n] * c_trial[n] * beta3,
```



```

        sigma);

    }
}

```

For the null model, just omit the term with `beta1` in both the model block and the generated quantities block. This is the model `pupil_null.stan`.

The models can be found in the `bcogsci` package:

```

pupil_cv <- system.file("stan_models",
                        "pupil_cv.stan",
                        package = "bcogsci")
pupil_null <- system.file("stan_models",
                          "pupil_null.stan",
                          package = "bcogsci")

```

Fit the models:

```

fit_pupil_int_ll <- stan(file = pupil_cv,
                        data = ls_pupil)
fit_pupil_int_null_ll <- stan(file = pupil_null,
                              data = ls_pupil)

```

Show summary of predictive accuracy of the models using the function `loo`.

```

(loo_int <- loo(fit_pupil_int_ll))

##
## Computed from 4000 by 41 log-likelihood matrix.
##
##           Estimate    SE
## elpd_loo   -251.0   5.3
## p_loo        5.2   1.5
## looic       502.1  10.5
## -----
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume MCMC draws (r_eff in [0.5, 1.4]).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.

```

```

(loo_null <- loo(fit_pupil_int_null_ll))

##
## Computed from 4000 by 41 log-likelihood matrix.
##
##           Estimate   SE
## elpd_loo    -255.3 4.6
## p_loo         4.4 1.1
## looic        510.7 9.3
## -----
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume MCMC draws (r_eff in [0.7, 1.2]).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.

loo_compare(loo_int, loo_null)

##           elpd_diff se_diff
## model1    0.0          0.0
## model2   -4.3          3.4

```

As it happened with the cloze probability effect in the previous section, we cannot decide which model has better predictive accuracy according to PSIS-LOO.

### 13 *K-fold-CV in Stan*

To use K-fold-CV (or LOGO-CV) in Stan (as opposed to PSIS-LOO), we need to be careful to store the log-likelihood of the *held-out data*, since we evaluate our model with only this subset of the data. The following example closely follows the vignette <https://cran.r-project.org/web/packages/loo/vignettes/loo2-elpd.html>.

The steps taken are as follows:

1. Split the data in 10 folds.

Since there is only one subject, we don't need to stratify (using `kfold_split_stratified()`), and we use `kfold_split_random()` from the `loo` package.

```
df_pupil$fold <- loo::kfold_split_random(K = 10, N = nrow(df_pupil))
# Show number of obs for each fold:
df_pupil %>%
  group_by(fold) %>%
  count() %>%
  print(n=10)

## # A tibble: 10 x 2
## # Groups:   fold [10]
##   fold     n
##   <int> <int>
## 1     1     4
## 2     2     4
## 3     3     4
## 4     4     4
## 5     5     4
## 6     6     4
## 7     7     4
## 8     8     4
## 9     9     4
## 10    10     5
```

2. Fit and extract the log pointwise predictive densities for each fold.

Compile the alternative and the null models first with `stan_model`, and prepare two matrices to store the predictive densities from the held out data. Each matrix has as many rows as post-warmup iterations in the models fit above ( $1500 \times 4$ ), and as many columns as observations in the data set.

```
pupil_stanmodel <- stan_model(pupil_cv)
pupil_null_stanmodel <- stan_model(pupil_null)
log_pd_kfold <- matrix(nrow = 6000, ncol = nrow(df_pupil))
log_pd_null_kfold <- matrix(nrow = 6000, ncol = nrow(df_pupil))
```

Next, loop over the 10 folds. Each loop carries out the following steps. First, fit each model (i.e., the alternative and null model) to all the observations except the ones belonging to the held-out fold using `sampling()`. As before, since we know the models converge, we use only

one chain for each model. The function `sampling()` uses the already-compiled models. Second, compute the log pointwise predictive densities for the held-out fold with `gqs()`. This function produces generated quantities based on samples from a posterior (in the `draw` argument) and ignores all the blocks except generated quantities.<sup>6</sup> Finally, store the predictive density for the observations of the held-out fold in a matrix by extracting the log likelihood of the held-out data. The output of this loop is a matrix of the log pointwise predictive densities of all the observations.

<sup>6</sup> The reader using `cmdstanr` rather than `rstan` might find a cryptic error here. This is because `cmdstanr` expects the parameters not to change. A workaround can be found in <https://discourse.mc-stan.org/t/generated-quantities-returns-error-mismatch-between-17869/15>

```
# Loop over the folds
for(k in 1:10){
  # Training set for k
  df_pupil_train <- df_pupil %>%
    filter(fold != k)
  ls_pupil_train <- list(c_load = df_pupil_train$c_load,
                        c_trial= df_pupil_train$c_trial,
                        p_size = df_pupil_train$p_size,
                        N = nrow(df_pupil_train))

  # Held out set for k
  df_pupil_ho <- df_pupil %>%
    filter(fold == k)
  ls_pupil_ho <- list(c_load = df_pupil_ho$c_load,
                     c_trial= df_pupil_ho$c_trial,
                     p_size = df_pupil_ho$p_size,
                     N = nrow(df_pupil_ho))

  # Train the models
  fit_train <- sampling(pupil_stanmodel,
                       iter = 3000,
                       chains = 1,
                       data = ls_pupil_train)
  fit_null_train <- sampling(pupil_null_stanmodel,
                            chains = 1,
                            iter = 3000,
                            data = ls_pupil_train)

  # Generated quantities based on the posterior from the training set
  # and the data from the held out set
```

```

gq_ho <- gqs(pupil_stanmodel,
             draws = as.matrix(fit_train),
             data = ls_pupil_ho)
gq_null_ho <- gqs(pupil_null_stanmodel,
                  draws = as.matrix(fit_null_train),
                  data = ls_pupil_ho)
# Extract log likelihood which represents
# the pointwise predictive density
log_pd_kfold[, df_pupil$fold == k] <-
  loo::extract_log_lik(gq_ho)
log_pd_null_kfold[, df_pupil$fold == k] <-
  loo::extract_log_lik(gq_null_ho)
}

```

### 3. Compute K-fold $\widehat{elpd}$ .

Next, evaluate the predictive performance of the model on the 10 folds using `elpd()`.

```

(elpd_pupil_kfold <- loo::elpd(log_pd_kfold))

##
## Computed from 6000 by 41 log-likelihood matrix using the generic elpd function
##
##      Estimate SE
## elpd   -244.4 1.5
## ic     488.8 3.1

(elpd_pupil_null_kfold <- loo::elpd(log_pd_null_kfold))

##
## Computed from 6000 by 41 log-likelihood matrix using the generic elpd function
##
##      Estimate SE
## elpd   -249.4 1.3
## ic     498.7 2.6

```

### 4. Compare the $\widehat{elpd}$ estimates.

```

loo_compare(elpd_pupil_kfold, elpd_pupil_null_kfold)

##      elpd_diff se_diff
## model1  0.0         0.0
## model2 -5.0         0.9

```

As with PSIS-LOO, we cannot decide which model has better predictive accuracy according to K-fold-CV.

## 14 *Summary*

In this chapter, we learned how to use K-fold cross-validation and leave-one-out cross-validation, using both built-in functionality in `brms` as well as `Stan`, in conjunction with the `loo` package. We saw an example of model comparison where cross-validation helped distinguish between the two models (log-normal vs. normal likelihood), and another example where no important differences were found between the models being compared (the N400 data with cloze probability as predictor). In general, cross-validation will be helpful when comparing rather different models [for examples from psycholinguistics, see Nicenboim et al 2018; and Yadav et al 2022]; when the models are highly similar, it will be difficult to distinguish between them. In particular, for typical psychology and linguistics data sets, it will be difficult to get conclusive results from model comparisons using cross-validation that aim to find evidence for the presence of a population-level (or fixed) effect, if the effect is very small and/or the data are relatively sparse (this is often the case, especially in psycholinguistic data). In such cases, if the aim is to find evidence for a theoretical claim, other model comparison methods like Bayes factors might be more meaningful.

## 15 *Further reading*

See the chapter in the textbook.